

CSC1029 OO Programming

Lecture 27 – Implementing Queues

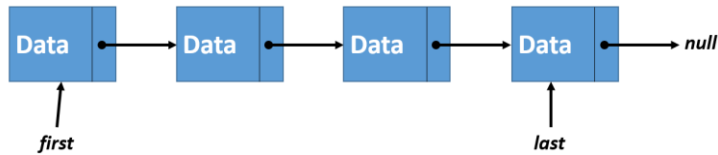


What's to come in today's lecture?

- Recap on Stack implementation
- Using a Linked List to Implement a Queue

A Stack Implementation (Recap)

SimpleStack
-myList: LinkedList
+SimpleStack() +push(Object):void +pop():Object +isEmpty():boolean

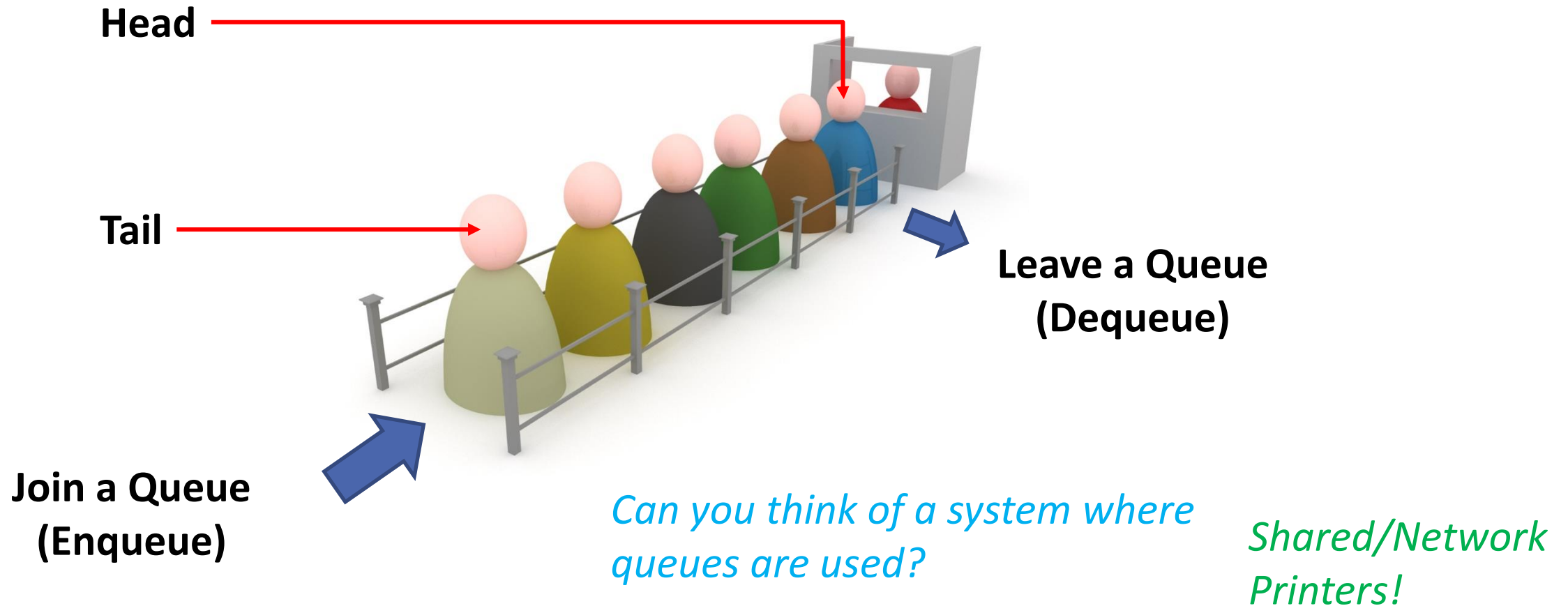


```
public static void main(...) {  
    SimpleStack myStack =  
        new SimpleStack();  
  
    myStack.push("One");  
    myStack.push("Two");  
    myStack.push("Three");  
    myStack.push("Four");  
    myStack.push("Five");  
  
    System.out.println();  
    String str = (String)myStack.pop();  
    while (str != null) {  
        System.out.println("Pop: " +  
            str + " from the stack.");  
        str = (String)myStack.pop();  
    }  
}
```

Pop: Five from the stack.
Pop: Four from the stack.
Pop: Three from the stack.
Pop: Two from the stack.
Pop: One from the stack.

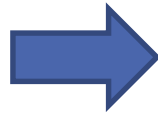
Last in .. First out
(LIFO)

A Queue Implementation



A Queue Implementation

SimpleQueue
-myQ: LinkedList
+SimpleQueue() +enqueue(Object):boolean +dequeue():Object +isEmpty():boolean +peek():Object



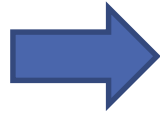
```
public class SimpleQueue {
    private LinkedList myList;

    public SimpleQueue() {
        myList = new LinkedList();
    }

    public boolean enqueue(Object obj) {
        // Join the back of the queue
        // Standard LinkedList add() will do the job
        if ( obj != null ) {
            myList.add(obj);
            return true;
        }
        return false;
    }
    ...
}
```

A Queue Implementation

SimpleQueue
-myQ: LinkedList
+SimpleQueue() +enqueue(Object):boolean +dequeue():Object +isEmpty():boolean +peek():Object



```
public Object dequeue() {  
    // Remove from the head of the queue  
    // LinkedList remove(0) used here  
    Object res = myList.remove(0);  
    return res;  
}  
  
public boolean isEmpty() {  
    return myList.size() == 0;  
}  
  
public Object peek() {  
    return myList.get(0);  
}
```

A Queue Implementation

SimpleQueue
-myQ: LinkedList
+SimpleQueue() +enqueue(Object):boolean +dequeue():Object +isEmpty():boolean +peek():Object

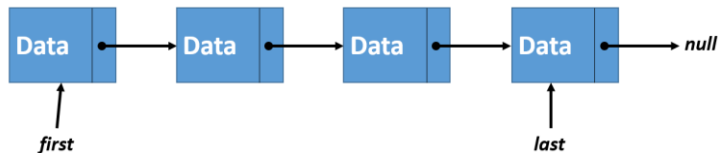
```
public static void main(String[] args) {  
    SimpleQueue myQueue = new SimpleQueue();
```

```
    myQueue.enqueue("One");  
    myQueue.enqueue("Two");  
    myQueue.enqueue("Three");  
    myQueue.enqueue("Four");  
    myQueue.enqueue("Five");
```

```
    System.out.println();  
    String str = (String) myQueue.dequeue();  
    while (str != null) {  
        System.out.println("Removed: " + str + " from  
                           the queue.");  
        str = (String) myQueue.dequeue();  
    }  
}
```

Removed: One from the queue.
Removed: Two from the queue.
Removed: Three from the queue.
Removed: Four from the queue.
Removed: Five from the queue.

First in .. First out
(FIFO)



What is Recursion?

Recursion is the process of defining a problem in terms of itself.

Recursion can apply to the **definition of functions** (methods) and to **data structures**.

Recursive methods are often used as an alternative to iterative (loop-based) solutions but care must always be taken to ensure that a recursive method terminates – in much the same way that we need to avoid infinite (non-terminating) loops.

A recursive solution is defined using **two** steps:

recursive step: a smaller version of the original problem

base case: a concrete answer to the smallest (basic) problem

What is Recursion?

A recursive solution is defined using **two** steps:

recursive step: a smaller version of the original problem

base case: a concrete answer to the smallest (most basic) problem

Consider the problem of counting (totalling) a list of integers:

- 1 Depending on how these integers are stored/managed, an
- 4 ***iterative solution*** would involve a ***loop***, adding each value in
- 3 turn to the overall total giving a final answer (15).
- 2 You will already be familiar with the kind of solution – usually
- 5 considered when looking at loops for the first time.

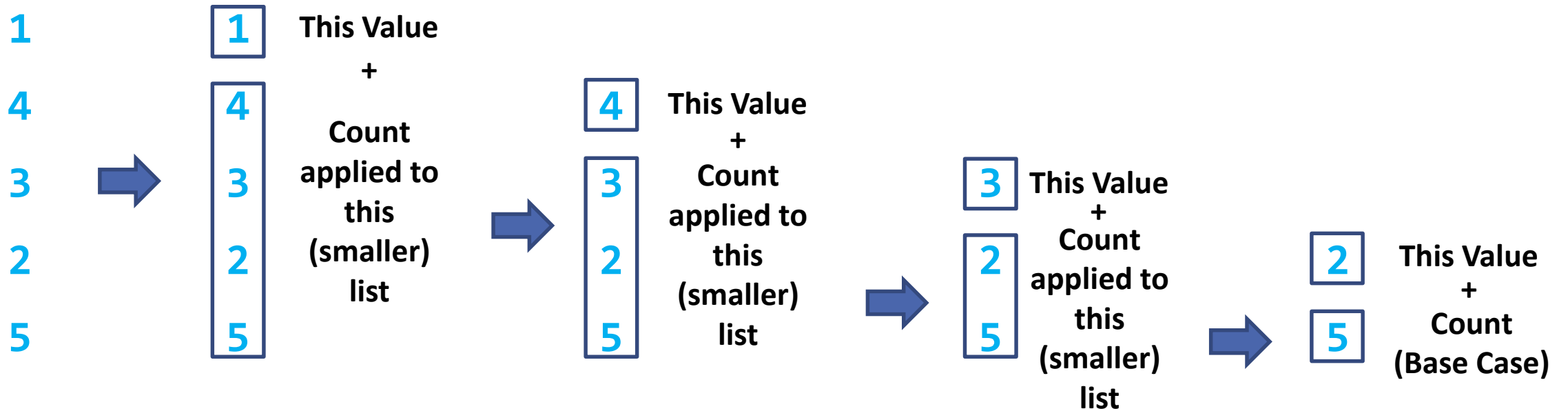
What is Recursion?

A recursive solution is defined using **two** steps:

recursive step: a smaller version of the original problem

base case: a concrete answer to the smallest (most basic) problem

Consider the problem of counting (totalling) a list of integers: ***A Recursive Solution***



Recursive Functions: Factorial

The **factorial** of a positive integer n (for $n \geq 0$, written as $n!$) is defined as:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 3 \times 2 \times 1$$
$$0! = 1$$

For example:

$$3! = 3 \times 2 \times 1$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

May be represented as a **function**:

```
public static int factorial(int n)
```

```
public static int factorial1(int num) {  
    //iterative calculation for num>=0  
    if ( num < 0 ) // return -1 for bad param  
        return -1;  
    if ( num == 0 ) // special case for 0!  
        return 1;  
    int res=1;  
    for(int value=num; value>=1;value--) {  
        res = res * value;  
    }  
    return res;  
}
```

```
int value = factorial(4);
```



24

Recursive Functions: Factorial

The **factorial** of a positive integer n (for $n \geq 0$, written as $n!$) is defined as:

$$n! = n \times (n-1) \times \dots \times 2 \times 1$$

$$0! = 1$$

For example:

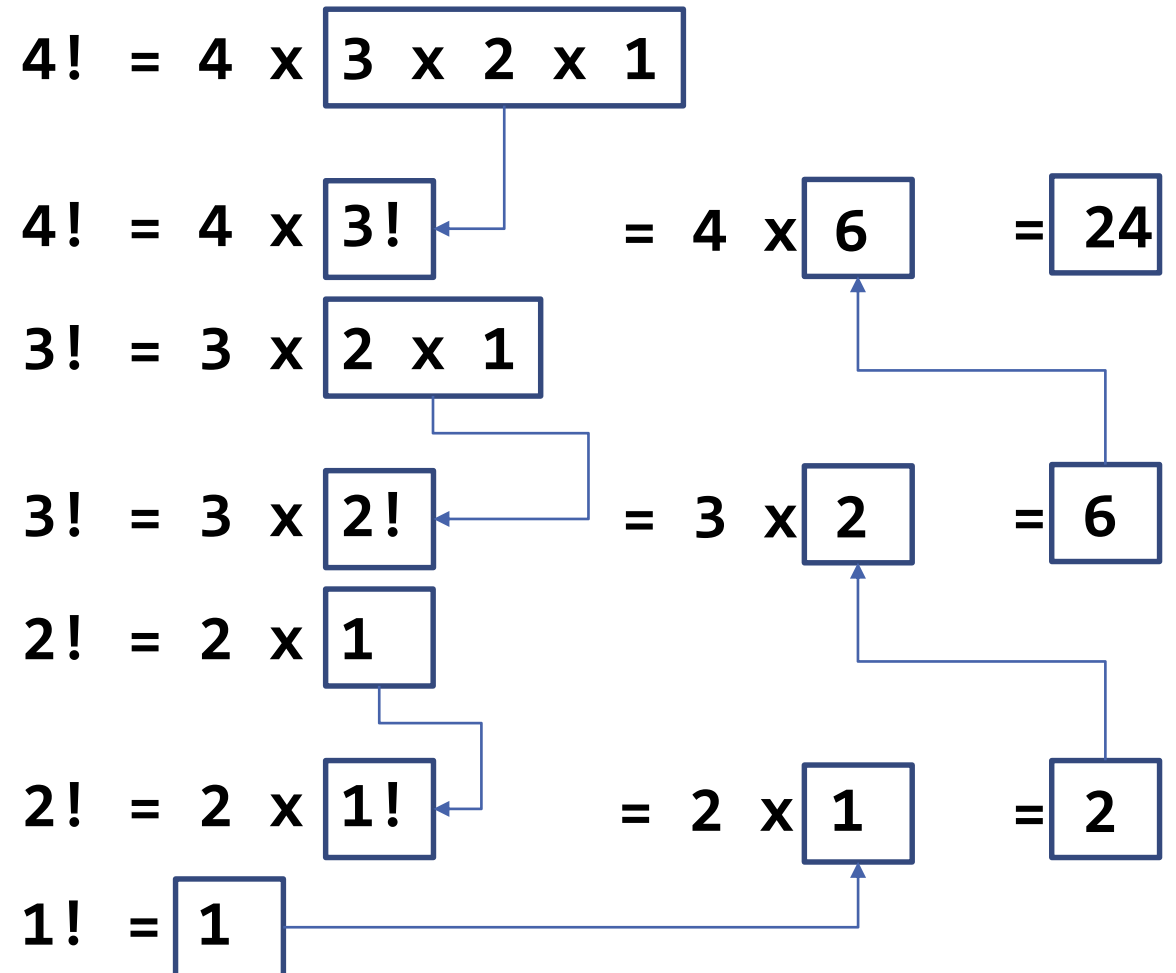
$$3! = 3 \times 2 \times 1$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

May be represented as a **function**:

```
public static int factorial(int n)
```

A recursive function is a function that is self-referential – i.e. defined in terms of itself.



Recursive Functions: Factorial

The **factorial** of a positive integer n (for $n \geq 0$, written as $n!$) is defined as:

$$n! = n \times (n-1) \times \dots \times 2 \times 1$$
$$0! = 1$$

For example:

$$3! = 3 \times 2 \times 1$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

May be represented as a **recursive function**:

```
public static int factorial(int n)
```

```
public static int factorial2(int num) {  
    // recursive calculation for num >= 0  
    // return -1 for bad parameter
```

```
    if ( num < 0 )  
        return -1;
```

```
    if ( num == 0 ) // base case  
        return 1;
```

```
    // recursive step  
    return num * factorial2(num-1);
```

```
}
```

Overall problem defined in terms of a **smaller version** of itself – terminating with a **base case**.

Recursive Functions: Palindromes

A fragment of text (String) is a palindrome if it reads the same way forwards and backwards.

May be defined as a function that returns a boolean value:

`isPalinrome(s) -> true if s == reverse(s)`

```
static boolean isPal(String s)
```

```
public static boolean isPal1(String s)
{ // non-recursive
  if ( s == null )
    return false;
  return s.equals(reverse(s));
}
```

```
private static String reverse(String s)
{
  String res="";
  for(int i=s.length()-1; i>=0; i--) {
    res += s.charAt(i);
  }
  return res;
}
```

Recursive Functions: Palindromes

A fragment of text (String) is a palindrome if it reads the same way forwards and backwards.

May be defined as a function that returns a boolean value:

```
isPalinrome(s) :  
    left = 0; right = len(s)-1  
    while(left<=right)  
        if ( s(left) != s(right) )  
            return false  
        left++; right--;  
  
    return true;
```

```
public static boolean isPal2(String data)  
{// non-recursive  
    if ( data == null )  
        return false;  
    int left=0;  
    int right = data.length()-1;  
    while(left<=right) {  
        if ( data.charAt(left) !=  
            data.charAt(right))  
            return false;  
    }  
    left++; right--;  
    return true;  
}
```

“a b b c b b a”

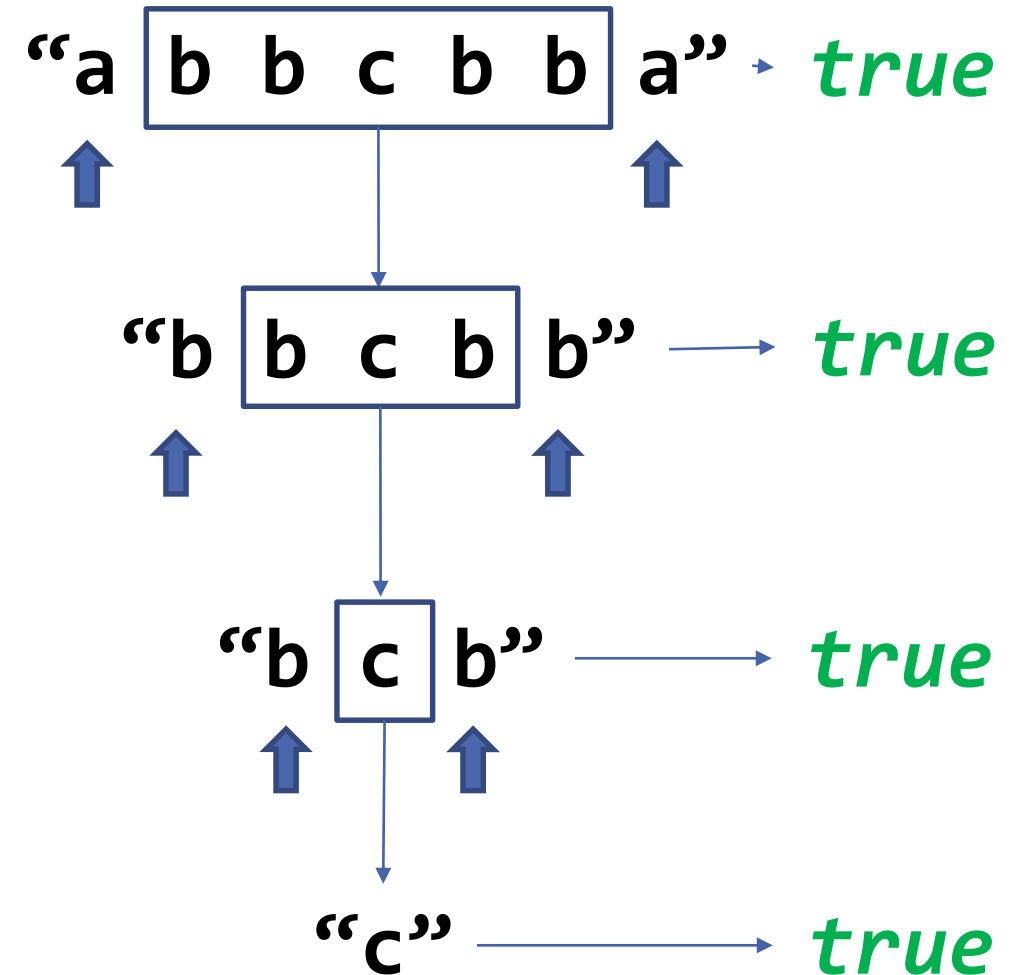
↑ ↑ ↑ ↑ ↑ ↑

Recursive Functions: Palindromes

A fragment of text (String) is a palindrome if it reads the same way forwards and backwards.

May be defined as a recursive function that returns a boolean value:

```
isPal(s) :  
  len = length(s)  
  if ( len==0 or len==1 )  
    return true;  
  if ( s(0) != s(len-1) )  
    return false;  
  return isPal(substring(1 to len-2))
```



Recursive Functions: Palindromes

A fragment of text (String) is a palindrome if it reads the same way forwards and backwards.

May be defined as a recursive function that returns a boolean value:

```
isPal(s) :  
    len = length(s)  
    if ( len==0 or len==1 )  
        return true;  
    if ( s(0) != s(len-1) )  
        return false;  
    return isPal(substring(1 to len-1))) }
```

```
public static boolean isPal(String data)  
{  
    if ( data == null )  
        return false;  
  
    if ( data.length() == 1 ||  
        data.length() == 0 )  
        return true; // base case  
  
    if ( data.charAt(0) !=  
        data.charAt(data.length()-1))  
        return false;  
  
    // recursive step  
    return isPal(data.substring(1,  
        data.length()-1));  
}
```

What's Next?

Revision & Prep for Assessment 3



What's Next?

Recursion & a Look Ahead to
Assessment 3